



The Iby and Aladar Fleischman Faculty of Engineering

Electronics - Laboratory B

FPGA Experiment Report - Stopwatch

Student 1:

Name: Saed Abu Fool

ID: 213851579

Student 2:

Name: Ameer Kais

ID: 325720001

Contents

1	Full Adder (FA)	4
1.1	Truth Table	4
1.2	Gate-Level Schematic	4
1.3	Simulation Waveform	5
2	Conditional Sum Adder (CSA)	6
2.1	Block Diagram	6
2.2	Timing Comparison with Ripple-Carry Adder	6
2.3	Generate Block	7
2.4	Simulation Waveform - CSA(3)	7
2.5	Simulation Waveform - CSA(4)	9
3	Limited Incrementor (Lim_Inc)	11
3.1	Block Diagram	11
3.2	Simulation Waveform - Lim_Inc(11)	11
3.3	Raw Sum vs Final Sum	12
4	Counter	14
4.1	Time Range	14
4.2	Utilizing Lim_Inc for Clock Division	14
4.3	Number of Lim_Inc Modules	14
4.4	Parameter L for 0.1 Hz Tick	14
4.5	50 MHz Clock Adaptation	14
4.6	Block Diagram	15
4.7	Simulation Waveform	15
4.8	External Register Requirement	16
4.9	Maximum Time Before Rollover	17
4.10	MM:SS Display Restructure	17
5	Stash	18
5.1	Block Diagram	18
5.2	Simulation Waveform	19
5.3	Sample Output Selection Logic	20
5.4	Alternative Approach	20
6	Control (Ctl)	21
6.1	Moore vs Mealy FSM	21
6.2	Block Diagram	22
6.3	Simulation Waveform	23
7	Debouncer	24
7.1	Block Diagram	24
7.2	Counter Value Tradeoff	25
8	Seven Segment Display (Seg_7_Display)	26
8.1	Clock Division Selection	26

9 Stopwatch	27
9.1 Block Diagram	27
9.2 Elaborated Design Schematic	29
9.3 Synthesized Design Comparison	30
10 Timing summary	32
Appendix: Verilog Source Code	33
.1 Verilog Implementations	33

1 Full Adder (FA)

1.1 Truth Table

Question: Write the truth table for the Full Adder, showing inputs a, b, ci and outputs sum, co. Which rows produce a carry out?

a	b	ci	sum	co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 1: Full Adder Truth Table

Rows that produce carry out: Rows with two 1's at least in the input will produce a high carry out, in our table the rows are: 4,6,7,8

1.2 Gate-Level Schematic

Question: Draw a schematic showing only AND, OR, XOR gates that implement your FA.

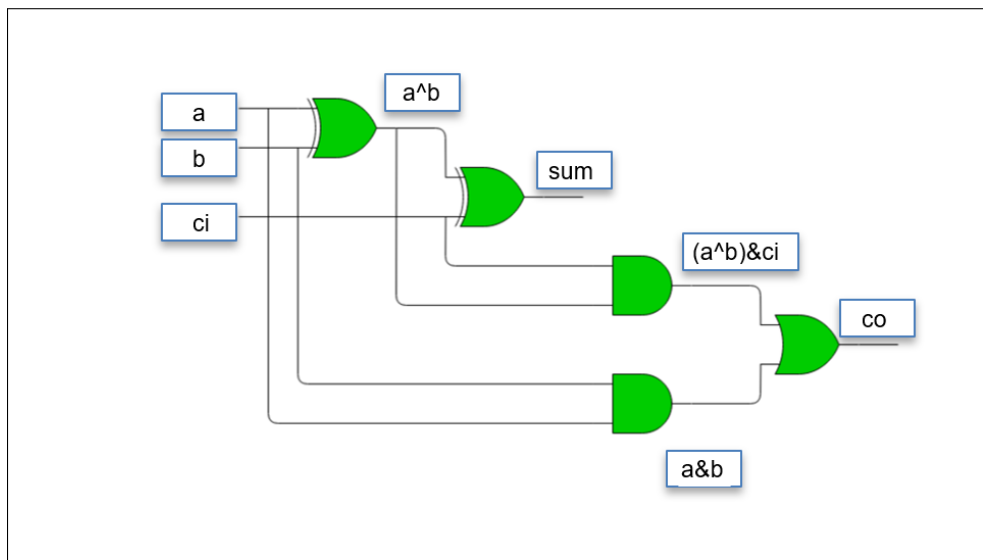


Figure 1: Full Adder Gate-Level Schematic

1.3 Simulation Waveform

Question: Provide an annotated waveform of your simulation.

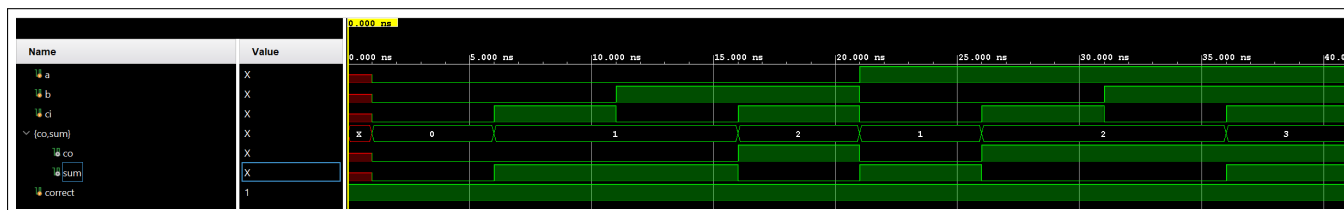


Figure 2: Full Adder Simulation Waveform (Annotated)

Waveform Explanation:

First of all, as we can see, correct stayed at '1' for the whole duration of the simulation indicating that the FA is working as intended. We added a bus that is made out of the concatenation of co and sum, which effectively gives us the output of the FA. As we can clearly see in all cases, co,sum equals the sum of the 3 inputs, which matches the truth table given above, and the general function of an FA module. In other words, x green rectangles in the current CC results in the number x in the results.¹

¹No annotation was added here since the simulation results are self explanatory, the result of the addition is under the operands directly and we can see everything without the need to add anything to the waveforms.

2 Conditional Sum Adder (CSA)

2.1 Block Diagram

Question: Design a block diagram for the module, use instantiated modules, show and explain any other logic necessary.

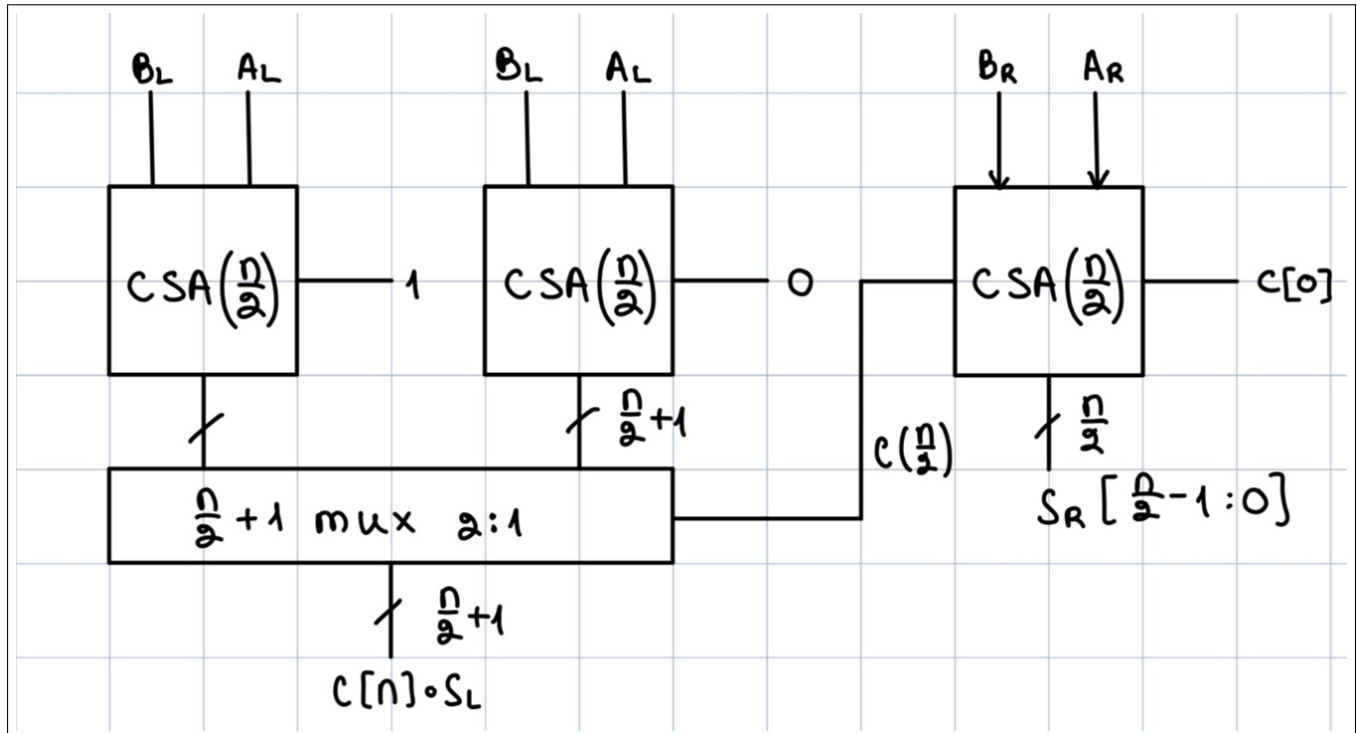


Figure 3: CSA Block Diagram

Block Diagram Explanation: This is the classic CSA block diagram that we have seen in the course "Digital Logic Systems". It adds parallelism to the addition operation, which in turn makes the operation faster, at the expense of space. Instead of waiting for each carry bit to be computed, as is done in an RCA for example, the second half of the addition (the left part of the bit string) is being computed in parallel, once assuming $c_i = 0$, and once assuming $c_i = 1$. In the end the right result is chosen via a multiplexer with the select bit being the co of the CSA responsible for the right part the bit string. This design is recursive, with $CSA(1)$ being the FA module we designed earlier.

2.2 Timing Comparison with Ripple-Carry Adder

Question: How does the Conditional Sum Adder improve timing compared to a simple ripple-carry adder? In what situations would a ripple-carry adder still be preferable?

Answer: In terms of timing, the CSA's critical path is no longer a full carry ripple through all N bits. The MSB-side sums are precomputed for both $c_i = 0$ and $c_i = 1$, so once the carry-out from the lower (right) block is available, the circuit only needs the MUX selection to choose the correct result. Thus, the worst-case delay is approximately the delay of the lower-block carry generation plus the MUX chain, rather than an N -stage carry-propagation chain as in an RCA.

A ripple-carry adder is still preferable when area, power, and simplicity are more important than speed. For small bit-widths the ripple delay is probably acceptable, so the extra duplicated logic and multiplexers of a CSA are not worth the cost. RCAs are also attractive in low-power/low-area designs and in non-critical timing paths, since they have minimal hardware, lower switching activity, and are easier to implement and verify (As seen in the previous section).

2.3 Generate Block

Question: What is a "generate block"? Why do we need to use it in the CSA implementation?

Answer: A generate block in Verilog is an elaboration-time construct (using `generate/endgenerate` with `for/if/case`) that lets the compiler replicate or conditionally include hardware based on parameters or constants. We need it in the CSA because the adder is built from repeated bit-slices/blocks (e.g., computing the sums for $c_i = 0$ and $c_i = 1$ across many bits). Instead of manually instantiating the same logic N times, a generate loop creates these repeated instances parameterically, making the design scalable to any bit-width and consistent across all stages (This explanation is general, and can be applied to any design that makes use of generate blocks).

2.4 Simulation Waveform - CSA(3)

Question: Provide an annotated waveform of your simulation. How can you confirm from the waveform that the "conditional" part of the design works correctly?

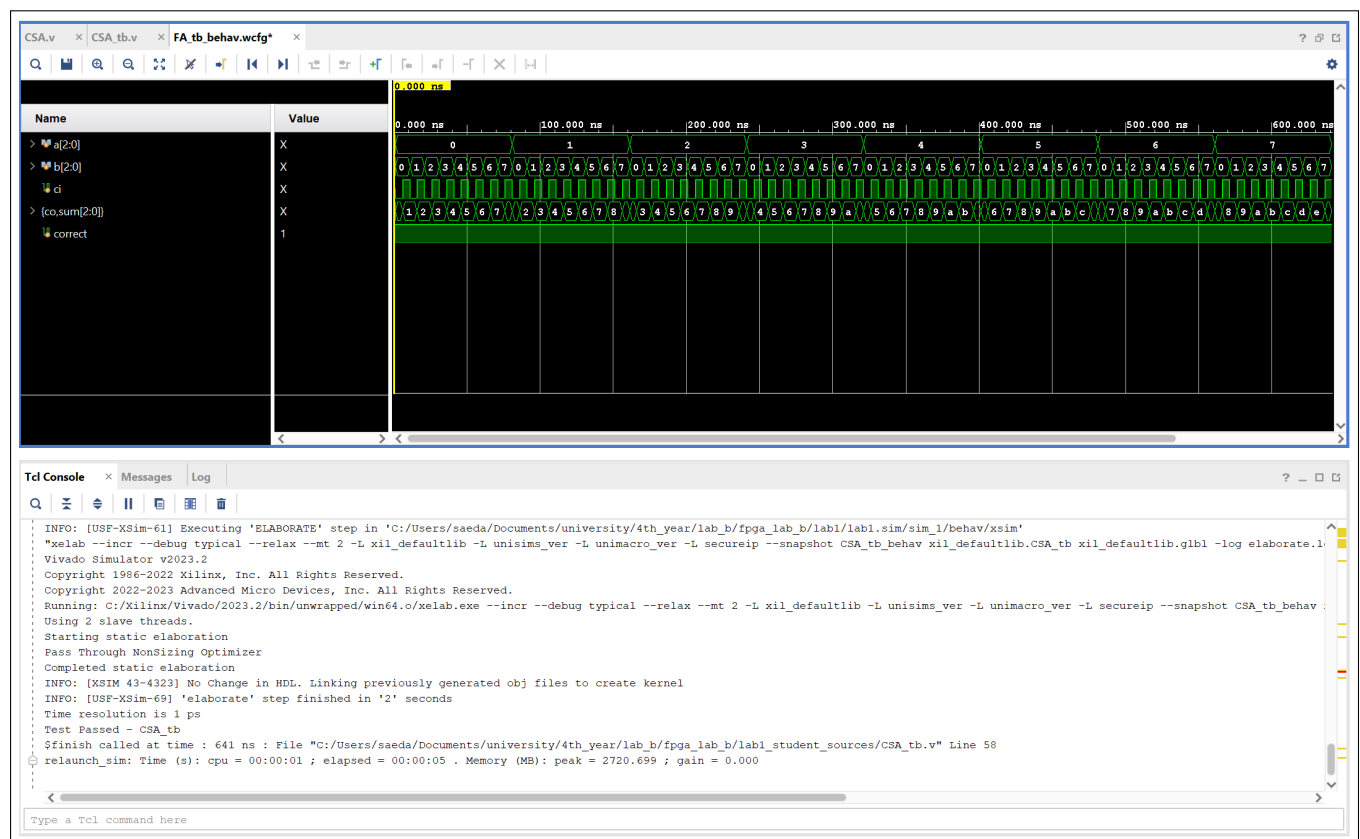


Figure 4: CSA(3) Simulation Waveform - Entire Simulation

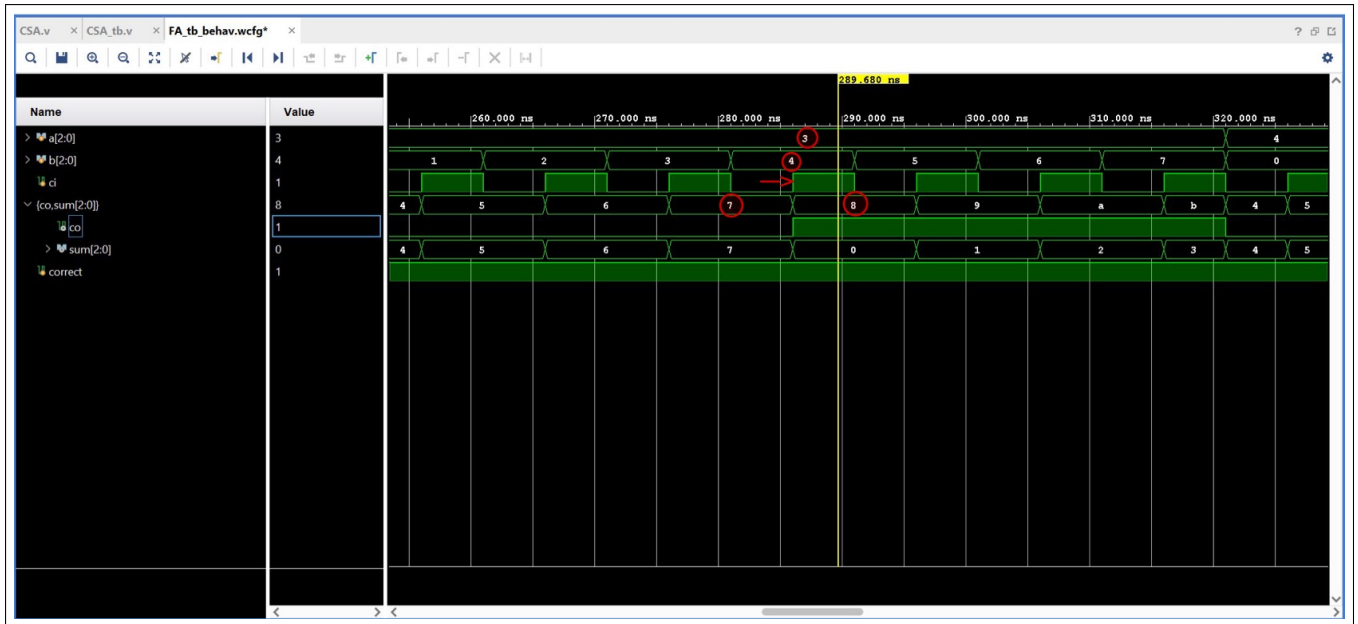


Figure 5: CSA(3) Simulation Waveform (Annotated)

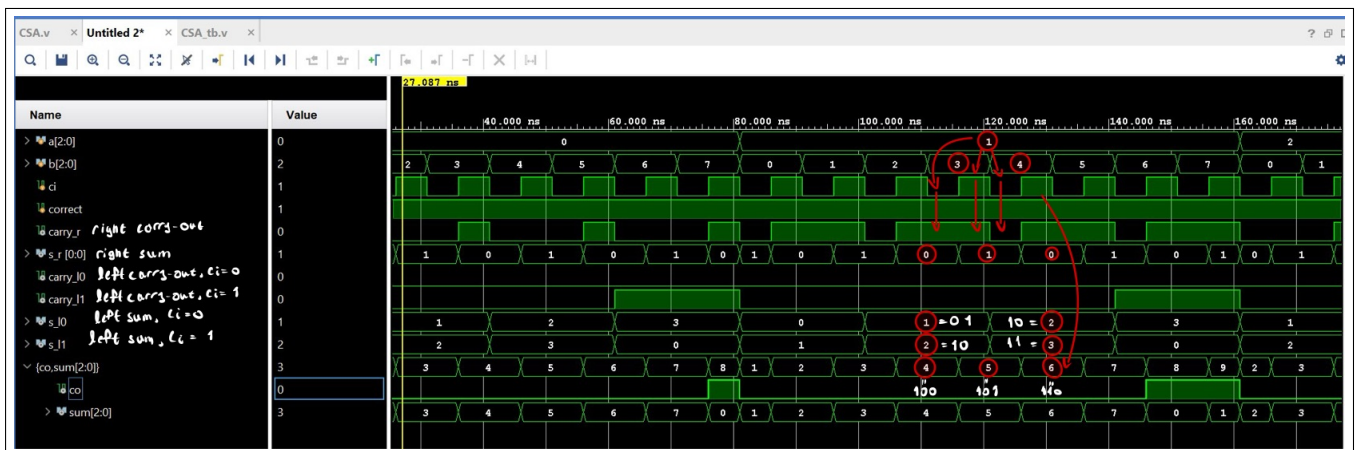


Figure 6: CSA(3) Simulation Waveform - Conditional Part (Annotated)

Waveform Explanation: All the relevant information regarding the signal names is given on the waveform.

First of all we can see that we got the Test Passed message in the console, and correct stays high in all the provided waveforms. Consider Figure 5: Looking at the case where $a = 3$, and $b = 4$, when $c_i = 0$ we get the result 7. When $c_i = 1$ we get the result 8. Our CSA is working as intended.

Now, in order to verify that the conditional part of the circuit is working, consider the waveform shown in figure 6: When $a = 1$, $b = 3$, and $c_i = 0$ the result is 4, and the sum of the right part of the CSA is 0. This means we want the left side to produce the number 2 (i.e. 10 in binary representation) so that the concatenation result would equal 4 (100 in binary representation). $carry_r = 1$ which means that the chosen left result should be the one with $c_i = 1$, (i.e. the result with the suffix l1). And as we can see the result is 4, which means l1 was chosen, exactly as is supposed to happen.

In the same case but with $c_i = 1$ we get the result 5, $carry_r = 1$, which means the

module chooses 11, but now the sum result of the right side equals 1, this means the we get $101_b = 5$ in the output, which is of course correct.

Now, when $a = 1$, $b = 4$ and $c_i = 0$ carry_r goes low, this means that the result of 10 is now chosen, which changes to $10_b = 2$ and when concatenated with the right result, we get 5 as expected.

In the same case, when $c_i = 1$ carry_r switches back to 1, now 11 is chosen. 11 outputs 3, concatenated with the result of the right side which is zero we get $110_b = 6$ which is exactly what we expected.

All in all, we can see that the conditional part of the circuit works as intended.

2.5 Simulation Waveform - CSA(4)

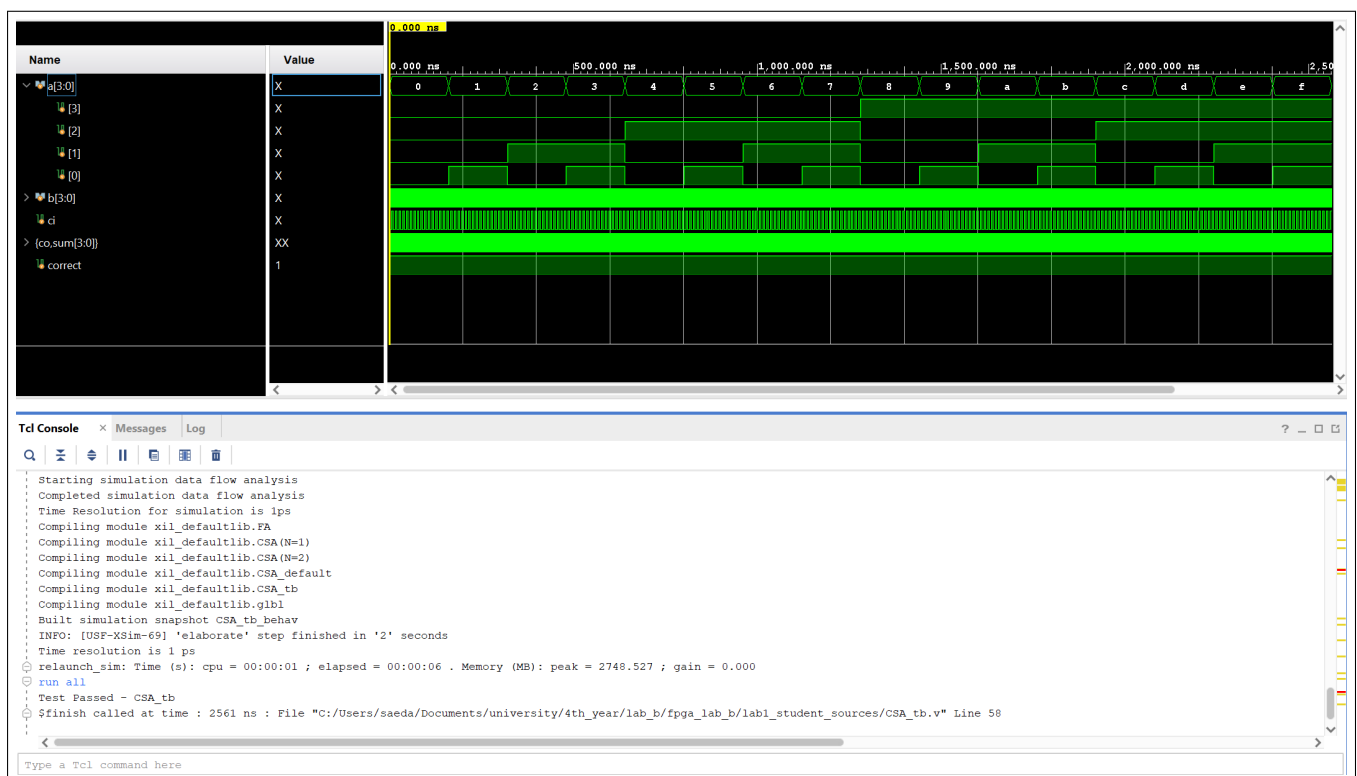


Figure 7: CSA(4) Simulation Waveform - Entire Simulation

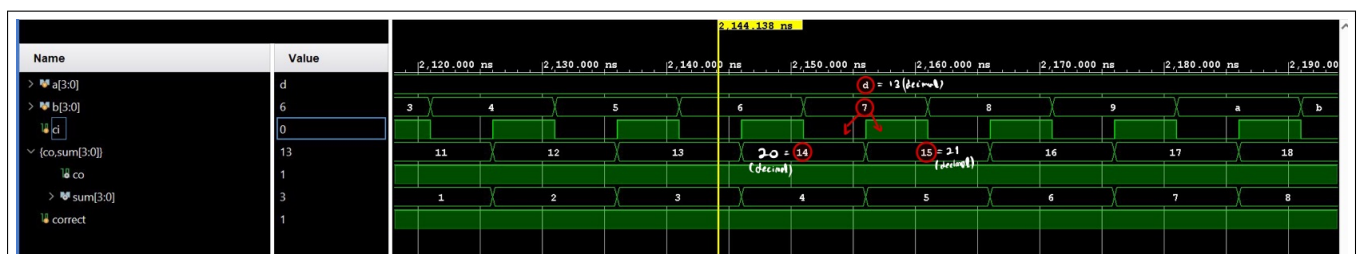


Figure 8: CSA(4) Simulation Waveform (Annotated)

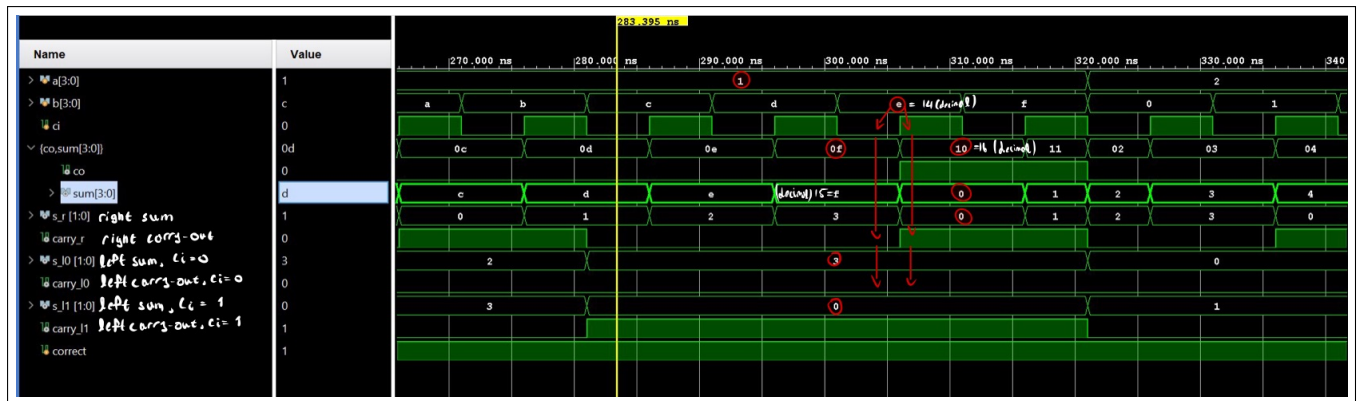


Figure 9: CSA(4) Simulation Waveform - Conditional Part (Annotated)

Waveform Explanation: We have updated the testbench, changing width parameter to 4.

We got the Test Passed message on the console, and correct stays at 1 till the end of the simulation.

Consider figure 8: $a = 13$ and $b = 7$. When $c_i = 0$ we get the result $14_{\text{hex}} = 20$, and when $c_i = 1$ we get $15_{\text{hex}} = 20$, which are both the results we expected. Our CSA works correctly after changing the N parameter to 4.

Now consider figure 9: Looking at the case when $a = 1$ and $b = e_{\text{hex}} = 14$, when $c_i = 0$ $\text{carry_r} = 0$, and we expect the module to choose l0's result = $011b = 3$, concatenating with the right part's result we get $011 \circ 11$ which equals $0F_{\text{hex}} = 15$ which is of course the right result. When $c_i = 1$ carry_r switches to 1, so l0's result is supposed to get chosen. l1 outputs 0, and the right part outputs 0, and l1 is chosen which is 10_b . The output is $1000_b = 16 = 10_{\text{hex}}$.

To summarize, the conditional part of the module works as intended, the module outputs the right result, and our parametric design works as intended when changing N.

3 Limited Incrementor (Lim_Inc)

3.1 Block Diagram

Question: Design a block diagram for the module, use instantiated modules, show and explain any other logic necessary.

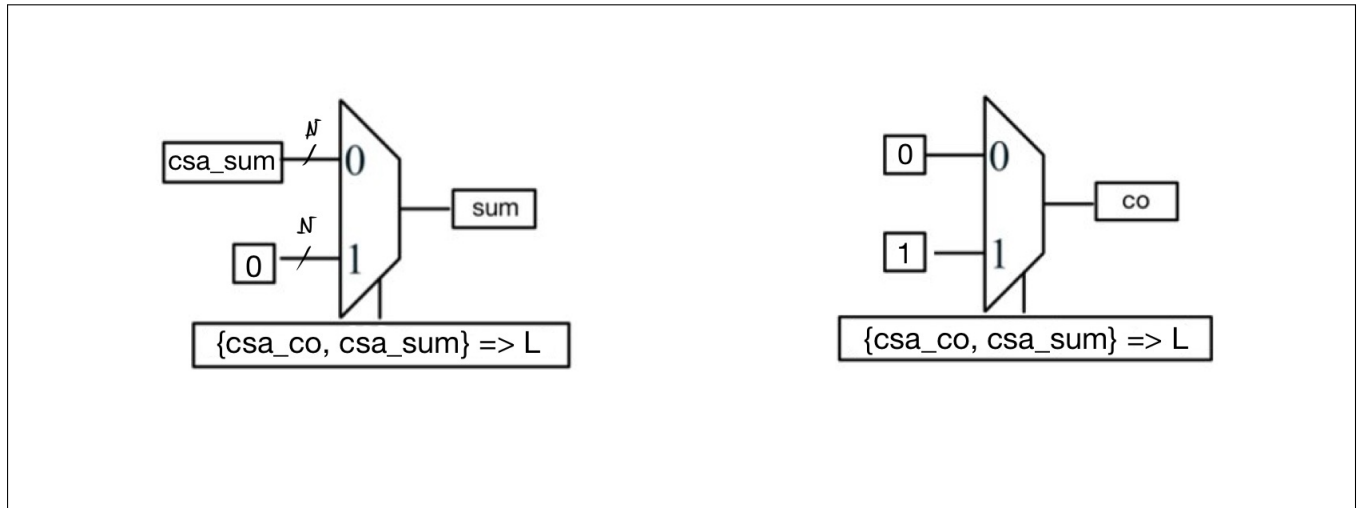


Figure 10: Limited Incrementor Block Diagram

Block Diagram Explanation: The block diagram is the spec giving to us, translated into hardware. The condition is the select bit of the 2 MUXes, and the output is chosen as given to us in the relevant section in the task document. The `csa_sum` and `csa_co` are the names of the outputs of the CSA that we used in order to compute the addition. Meaning, the limited incrementer is made up of 2 MUXes and a CSA.

3.2 Simulation Waveform - Lim_Inc(11)

Question: Provide an annotated waveform of your simulation. Point to one case where CSA's "raw sum" differs from the final sum.

Important Note: We were asked to change the parameter `L` from 7 to 11 in the course forum.

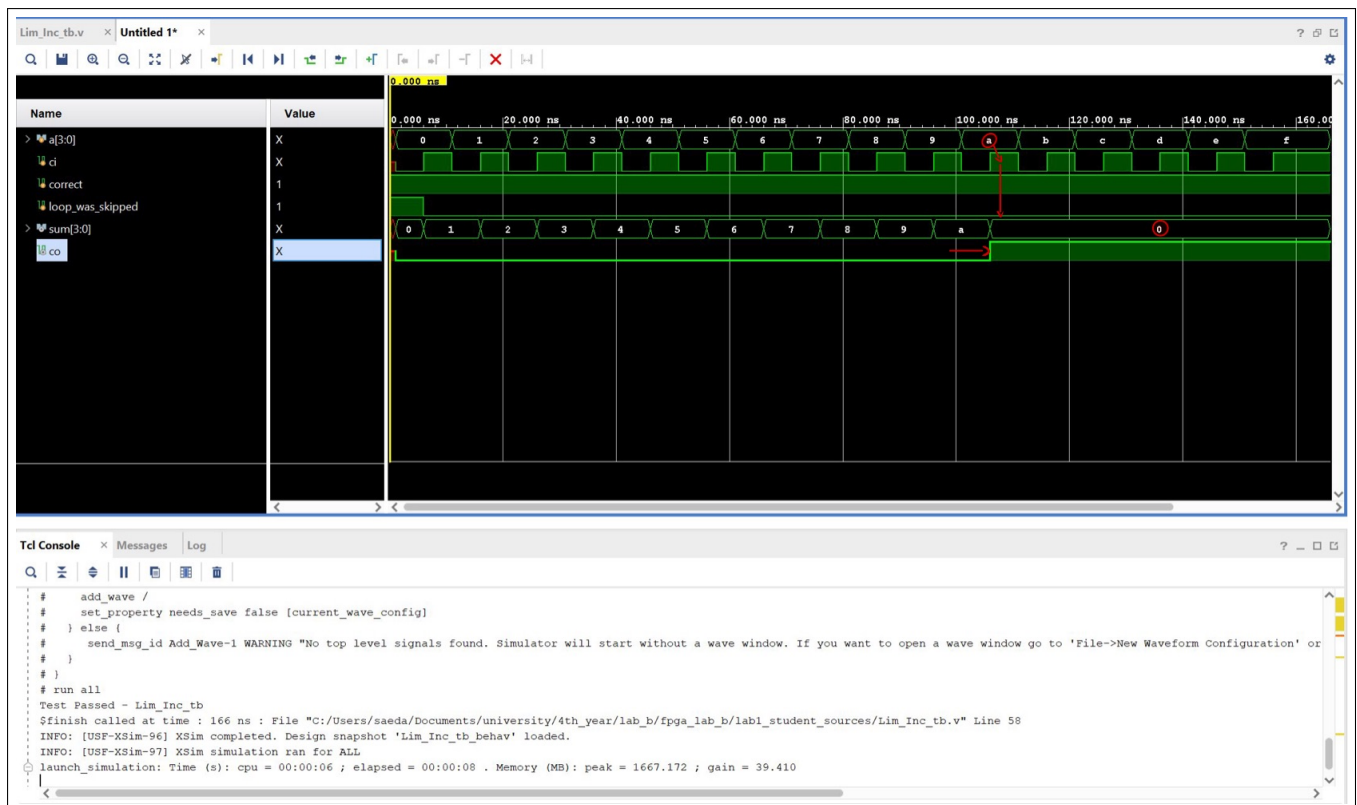


Figure 11: Lim_Inc(11) Simulation Waveform - Full (Annotated)

Waveform Explanation: As we can clearly see, we got the Test Passed message, and correct stays high until the end of the simulation. Knowing that the sum is between c_i and a , the interesting part would be when $a = 10$ (i.e. a in hexadecimal representation). When $a = 10$ and $c_i = 1$, the final sum equals $1000_b = 16$, which does not equals the raw sum which of course equals 11.

3.3 Raw Sum vs Final Sum

Question: Explain why the "raw sum" differs from the outputted sum and co . Is there a case where they never differ? If so, elaborate on that case and suggest a simpler design that would be sufficient in that situation.

Answer: This happens because we designed the module to "act" this way, the raw sum is now equal (or larger than) to L , based on the functionality of the module, it should output 10000, and it does, as expected.

The funny case that would make this limited incremeter just a regular incremeter is when L equals a power of 2, i.e. $L = 2^N, N \in \mathbb{N}$.

In this case $\{cas_co, cas_sum\} \neq L$, which means an alternative module in that case can be just a CSA(N).

Another waveform screenshot is added below to show the CSA raw sum in order to visualize what was said in previous sections.

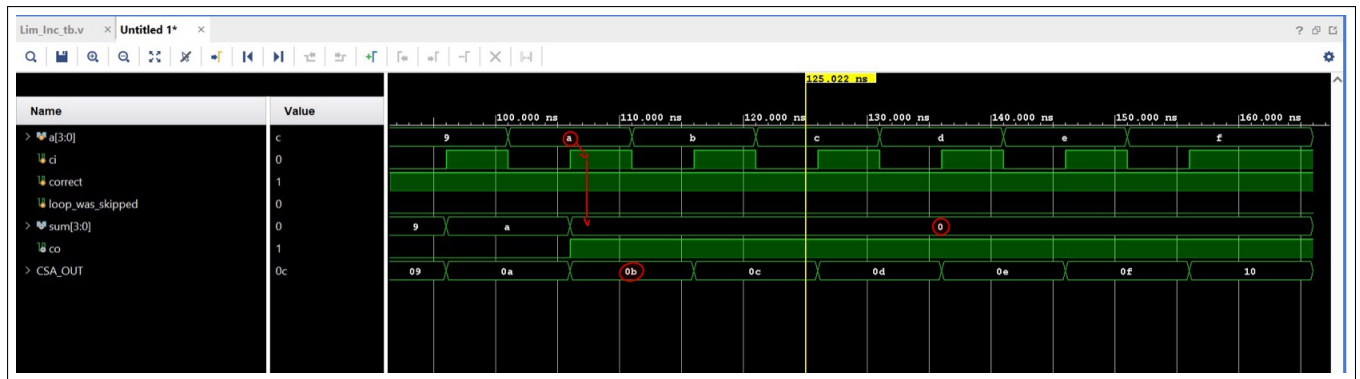


Figure 12: Lim_Inc(11) Simulation Waveform - Raw vs Final

4 Counter

4.1 Time Range

Question: The output `time_reading[7:0]` represents the time in seconds. Specify the time range it can represent (sec).

Answer: Based on the spec given to us in the ask document, `time_reading` can represent numbers from 0 to 99, making that the time range of seconds it can display.

4.2 Utilizing `Lim_Inc` for Clock Division

Question: Describe how can you utilize a `Lim_Inc(L)` module and a register in order to toggle the counting of the seconds when fed by a 100MHz clock?

Answer: We can use a limited incrementer with $L = 10^8$, this essentially gives us a threshold of 100M ticks which mean a second has passed. We'd then connect the output of the incrementer to a register, and the output of the register to the a input of the limited incrementer. This means each cycle a would increase by 1 until it reaches $L = 10^8$, which would indicate a second has passed and the carry out bit would go high, which would be used as the indicator.

4.3 Number of `Lim_Inc` Modules

Question: What determines how many `Lim_Inc` modules you need in the counter?

Answer: How many digits we want to represent, for example in our case here we want to count 2 digits (up to 99) then we need 1 limited incrementer to indicate a second has passed, another one to actually count the seconds and another one to count the daseconds. All in all we need the number of digits + 1.

4.4 Parameter L for 0.1 Hz Tick

Question: How do you choose the parameter L for the divider that generates the 0.1 Hz tick?

Answer: There can be a few ways to achieve this, if we have a limited incrementer that indicates 1Hz ticks, in the next stage we can add another `lim_inc` with $L = 10$. That would count the seconds and indicate the 0.1Hz tick. We'd then chain that with a register and another `lim_inc` with $L = 10$, and that would count the daseconds, and would rollover to zero when it reaches 100 seconds.

Another solution can be having another limited incrementer with $L = 10^9$ and have chained with another limited incrementer (with $L = 10$) that would count it (with a register of course). That way the first incrementer would indicate the ticks, and the second stage would count it, similar to the solution of the previous section.

We went with the first solution since we use 1 less `Lim_Inc` (3 instead of 4).

4.5 50 MHz Clock Adaptation

Question: If the input clock was 50 MHz instead of 100 MHz, how would your parameter values or design connections change?

Answer: Using our solution, we'd need to change the first `Lim_Inc`'s L to 5×10^7 and everything else would stay the same.

4.6 Block Diagram

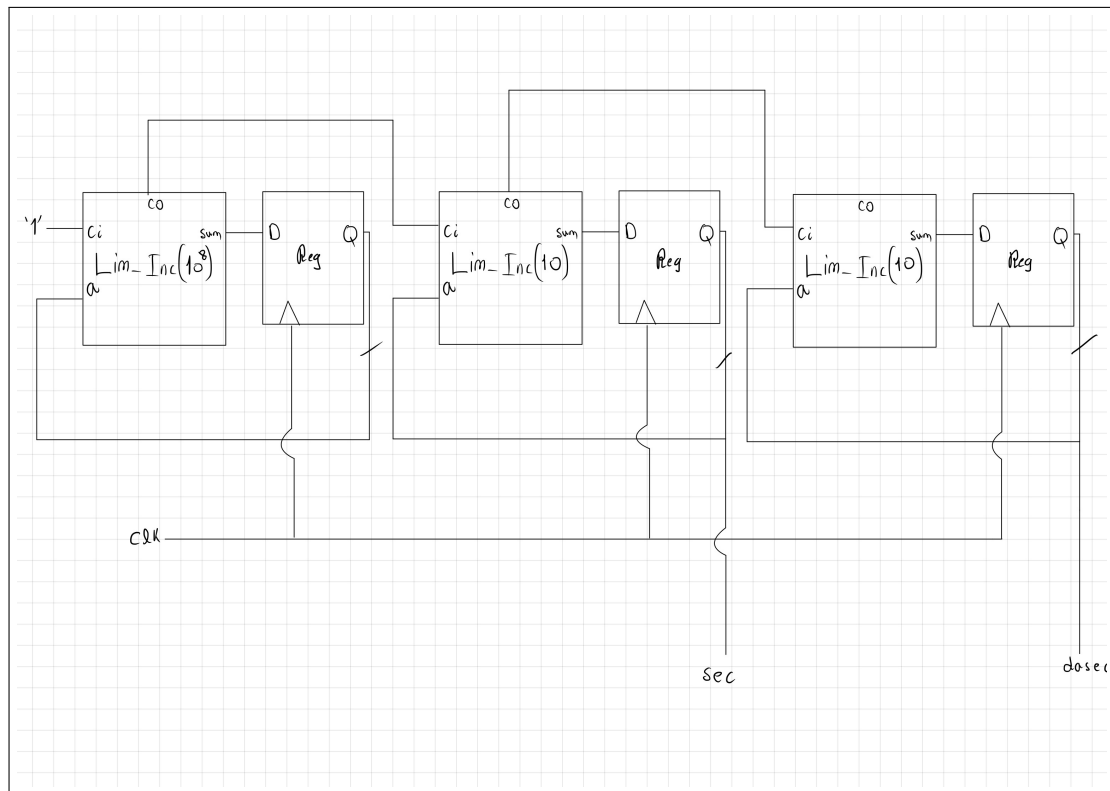


Figure 13: Counter Block Diagram

Block Diagram Explanation: As explained above, we have 3 limited incrementers. The first one ticks when 1 second has passed. The second one counts until it reaches ten seconds then rolls over. And the last stage counts until it reaches 10 daseconds and rolls over. The Carry-Out bit of each stage is used to start the count of the next stage.

4.7 Simulation Waveform

Question: Provide an annotated waveform of your simulation (zoom into an interesting section, where at least 2 digit changes should be observed).

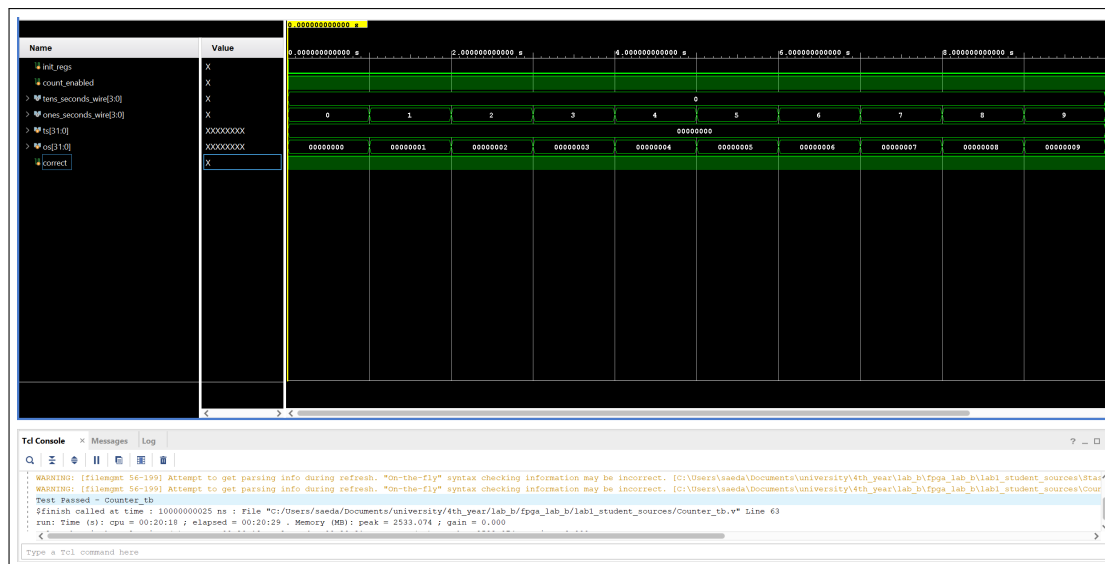


Figure 14: Counter Simulation Waveform - Full Simulation

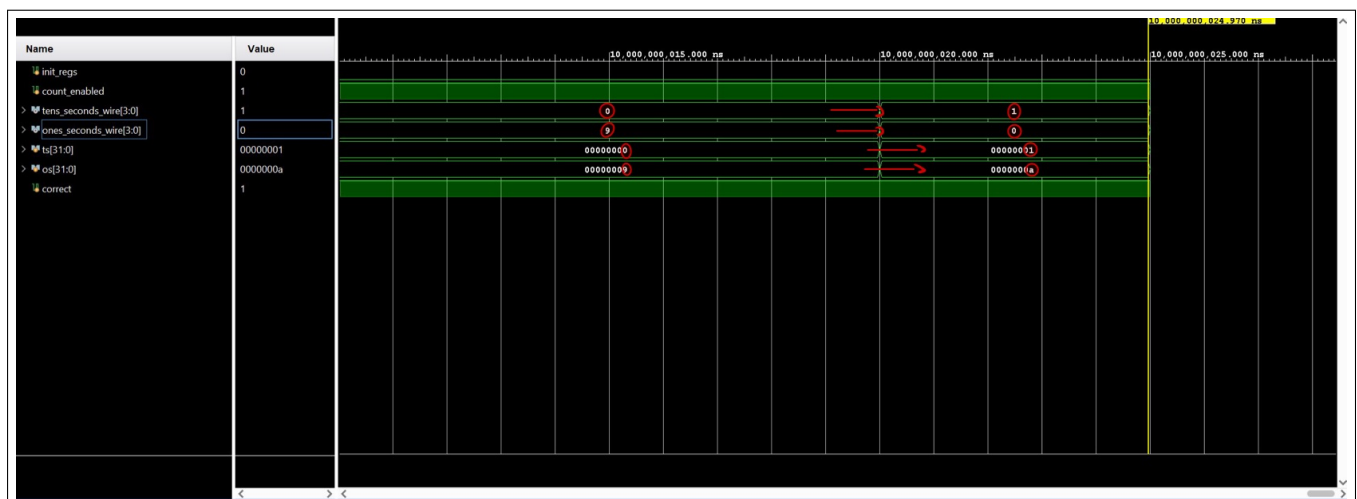


Figure 15: Counter Simulation Waveform - Switch from 09 to 10 Seconds (Annotated)

Waveform Explanation: As we can see, we got the Test Passed message in the console, and correct stays high throughout the whole simulation.

Looking at Figure 14 we can see the counter works as intended.

If we look at Figure 15 we can see the switch from 9 seconds to 10 seconds, which matches the counters set in the testbench (os and ts), and the timeline in the simulation (apart from the 2 cycle delay we added in the beginning before starting the count).

4.8 External Register Requirement

Question: Why can't the Lim_Inc module hold its own count internally (i.e., why must we use an external register)?

Answer: The Lim_Inc module is a purely combinational module, so it cannot store state. Hadn't we used a register, we'd need to connect its output to its input, which makes a combinational loop. In hardware, this would cause the output to change according to the circuit's propagation delays rather than on clock edges, which is not the

intended counter behavior. Moreover, Verilog modules containing combinational feedback loops aren't reliably synthesizable, and if we tried to simulate a circuit like this to test its functionality in a zero-delay RTL simulation, we would get unpredictable/unreliable behavior. All in all, this is a very bad idea.

4.9 Maximum Time Before Rollover

Question: What is the maximum time the counter can represent before rolling over?

Answer: The counter can count up to 99 seconds before rolling back over to 00.

4.10 MM:SS Display Restructure

Question: If you wanted the module to display minutes and seconds (MM:SS) instead of only seconds, how would you restructure your `Lim_Inc` hierarchy? Which parameters or additional modules would you need?

Answer: The main structure would mostly remain the same, the first stage would stay the same, but the 2 next stages would count a bit differently:

The 2nd and 3rd stages would count up to 60 i.e. $L = 60$. Which means we need larger limited incrementers and larger registers to store the count.

This counter can count up to 59:99 (MM:SS) and would roll over after an hour.

5 Stash

5.1 Block Diagram

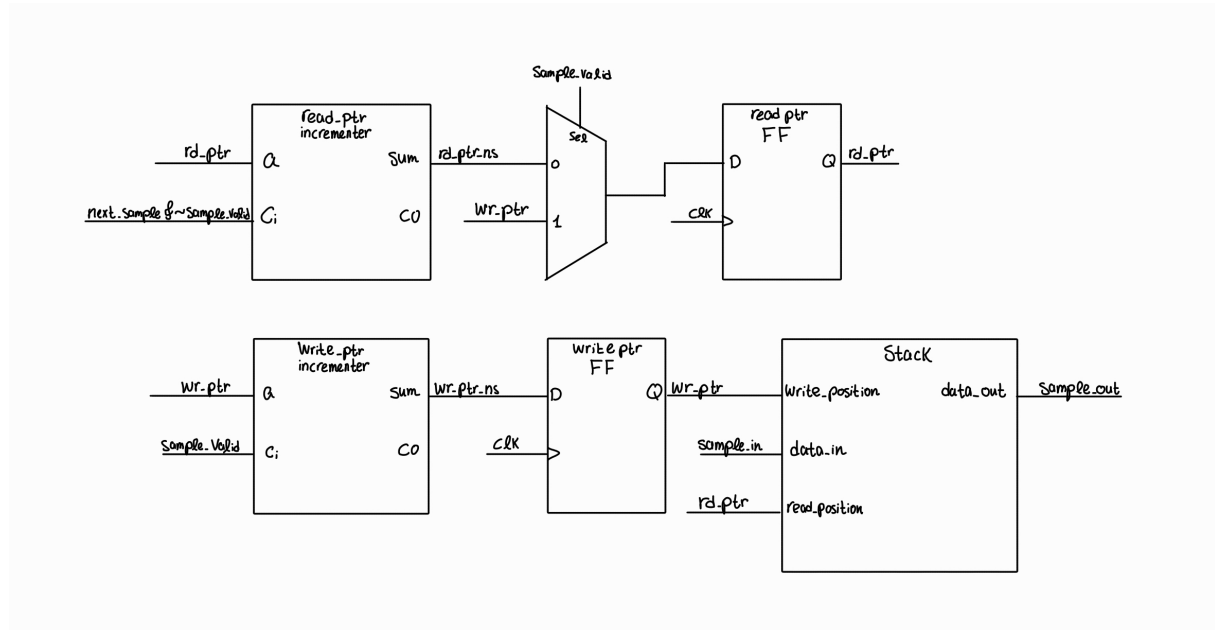


Figure 16: Stash Block Diagram

Block Diagram Explanation: In the block diagram there are 5 main blocks:

1. read_ptr incrementer: Lim_inc instance that can count up to "DEPTH", it increments the read pointer each time the module recives a high next_sample signal.
2. write_ptr incrementer: Lim_inc instance that can count up to "DEPTH", it increments the write pointer each time the module recives a high sample_in_valid signal.
3. write_ptr FF: flip-flop that saves write_ptr incrementer outputs.
4. read_ptr FF: flip-flop that saves the read_ptr, its input depends on the sample_in_valid signal:

$$\text{input} = \begin{cases} \text{read_ptr_ns}, & \text{sample_in_valid} = 0 \\ \text{write_ptr}, & \text{sample_in_valid} = 1 \end{cases}$$

5. Stack: a 2D-array, contains "DEPTH" registers, each is 8-bits. This is used to save the samples. When writing, the module accesses the appropriate register by using the write pointer, which indicates where the module should save the sample. When reading, the module accesses the appropriate register by using the read pointer, which indicates which register we should expose its output.

5.2 Simulation Waveform

Question: Provide an annotated waveform of your simulation. Show in your waveform a cycle where `sample_in_valid=1` and `next_sample=0`, and explain why `sample_out` shows the new value immediately.

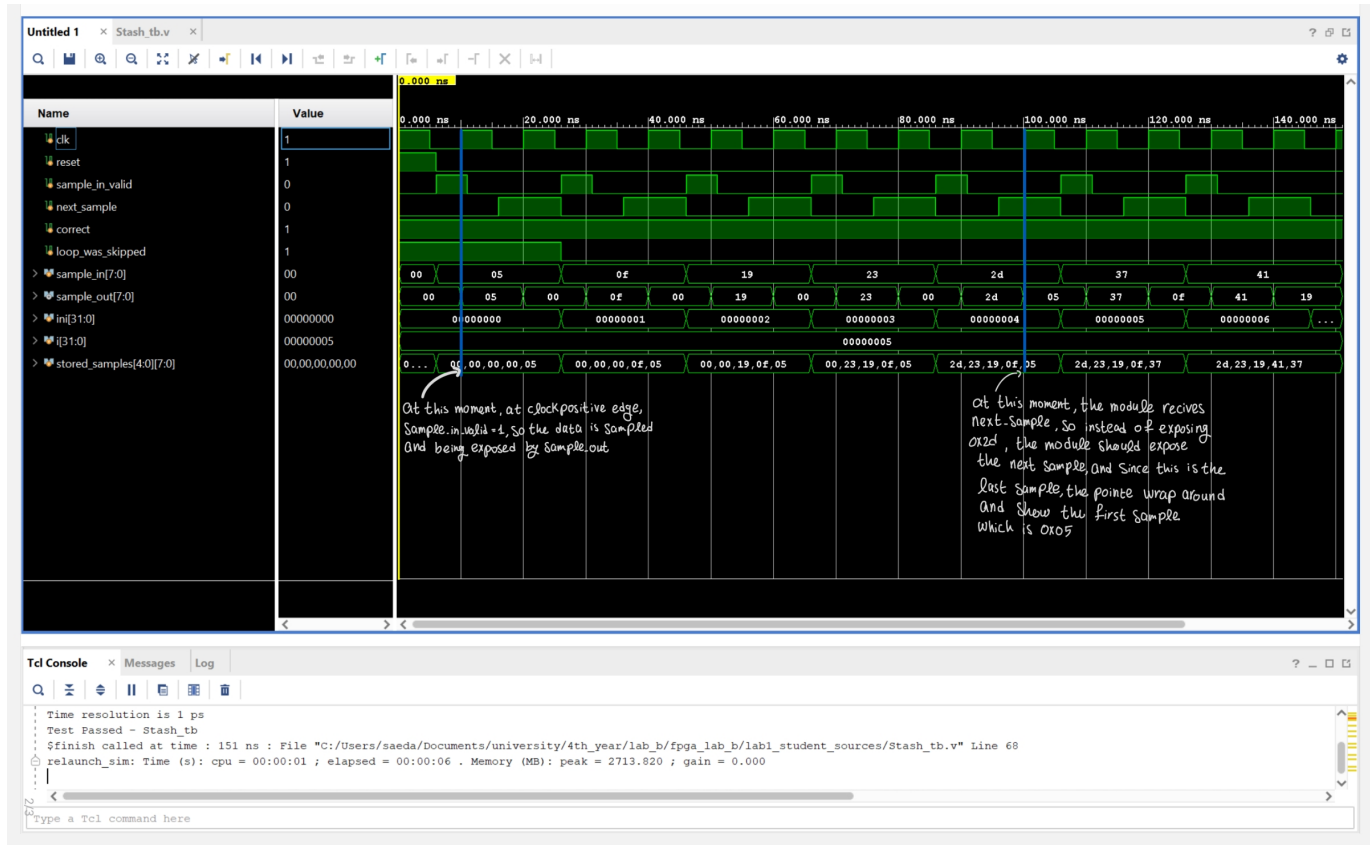


Figure 17: Stash Simulation Waveform (Annotated)

Waveform Explanation: As we can see in the console, the test passed.

When receiving `sample_in_valid` signal, the module samples the inputs and saves it into the stack, to the appropriate register according to the write pointer, we can see that the stack is being filled in order, and once it's full the pointer wraps around and replaces the oldest sample with the new one.

When receiving `next_sample` signal, the module exposes the next sample, and when reaching the final one, it wraps around so the next `next_sample` is the first one.

All the read/write operations are performed synchronously (Update on a positive edge of the clock input), as the instructor requested in the lab forum.

5.3 Sample Output Selection Logic

Question: Explain how you implemented the `sample_out` selection logic.

Answer: In our design we have a read pointer, which indicates the position we wish to read, the logic is:

$$\text{sample_out} = \text{stack}[\text{read_ptr}]$$

`read_ptr` itself has logic to decide when and what to read. When a `sample_in_valid` signal is asserted, the read pointer follows the write pointer so the `sampled_in` value is exposed immediately. Otherwise, whenever a `next_sample` signal is received, using a limited incrementer, the read pointer is incremented by one and once it is pointing at the last sample, `next_sample` signal will make it back to the first sample.

5.4 Alternative Approach

Question: Propose another different approach to the `sample_out` immediate visibility logic, compare pros and cons between the two approaches.

Answer: Another logic we can use is:

$$\text{sample_out} = (\text{sample_in_valid}) ? \text{sample_in} : \text{stack}[\text{read_ptr}]$$

Pros and cons for each design:

1. Our design approach:
 - (a) Pros: `sample_out` always comes from the same logic $\text{sample_out} = \text{stack}[\text{read_ptr}]$. since `read_ptr` is changing synchronously (register output), `sample_out` will be synchronous also.
 - (b) Cons: Requires more logic for `read_ptr` to meet the required specifications
2. new design approach:
 - (a) Pros: Simple implementation and more intuitive.
 - (b) Cons: `sample_out` will not be synchronized, it will change whenever the module receives `sample_in_valid` signal.

6 Control (Ctl)

6.1 Moore vs Mealy FSM

Question: The FSM is presented as a Mealy state diagram. Explain whether this FSM is a Moore FSM.

Answer: The original FSM is depicted as a Mealy machine, its outputs depend on both the current state and the inputs. This is evident when we look at the state diagram: the arcs between states are labeled with both inputs and outputs (in the format input/output). In a typical Moore machine, on the contrary, the outputs depend only on the current state and not on the inputs. Looking closely at the state diagram, we can see that the output only depends on the current state, as the change in inputs has no effect on them. This means that we can rewrite the state machine as a Moore state machine, without changing its behavior.

The new state machine in Moore representation is given below:

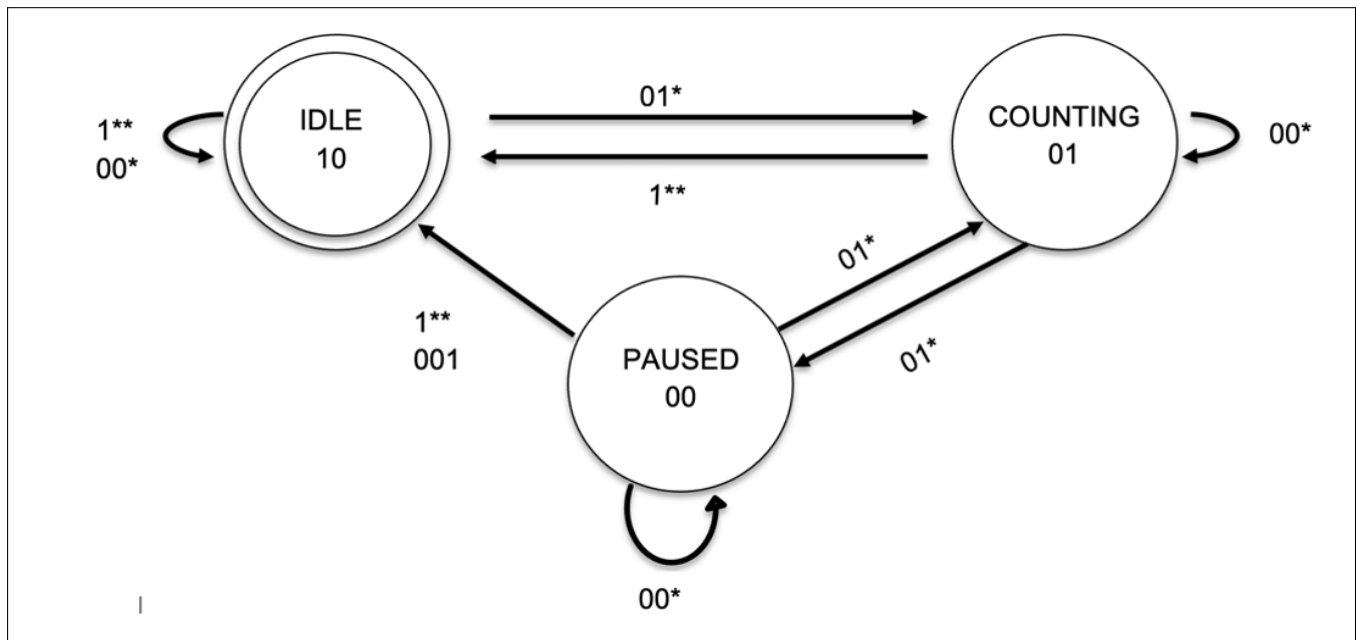


Figure 18: Control FSM - Moore Representation

With the outputs {init_regs, count_enabled} displayed on each state.

6.2 Block Diagram

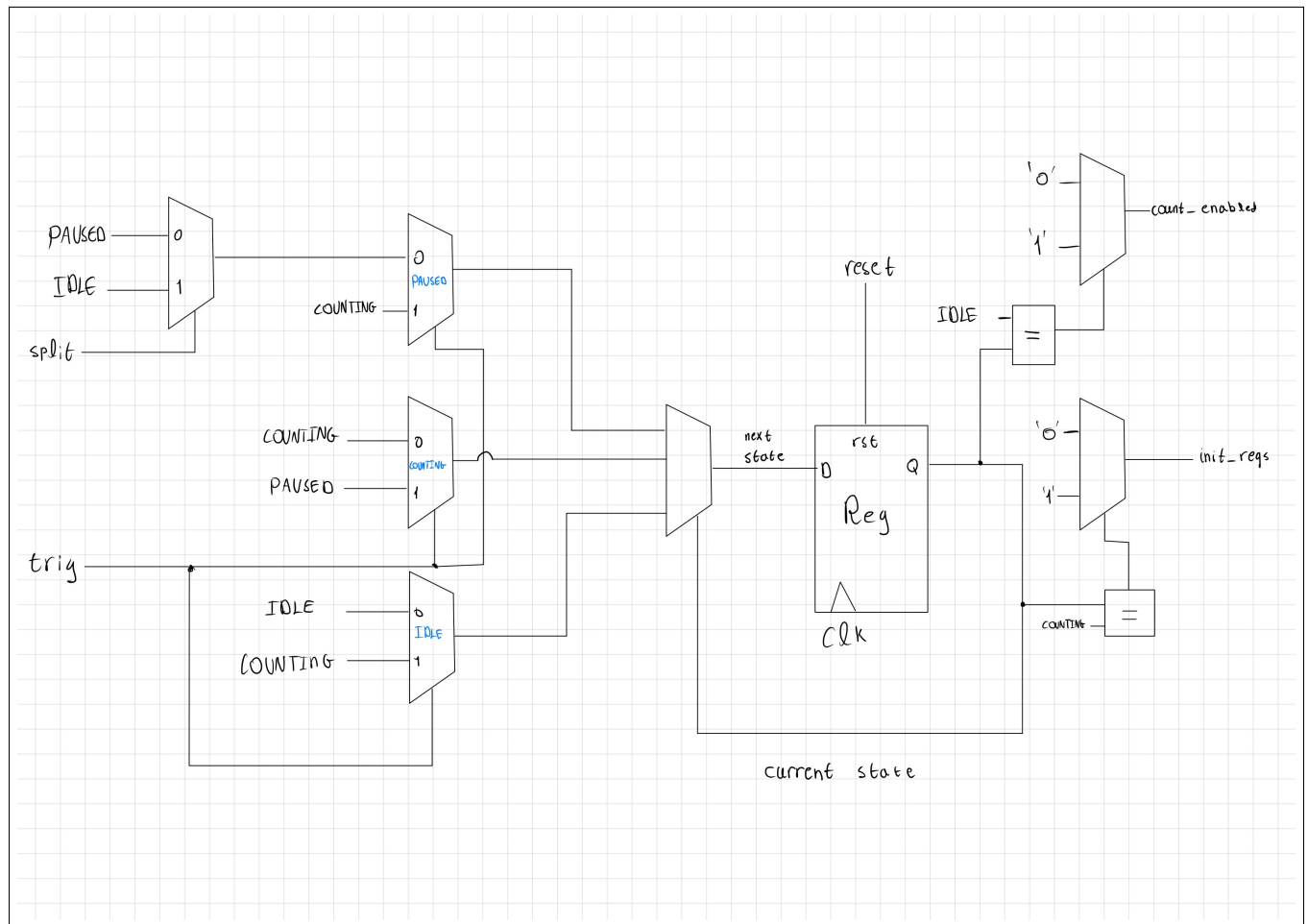


Figure 19: Control FSM Block Diagram

Block Diagram Explanation: As we can see, this block diagram shows how this is a clear Moore state machine. The outputs depends solely on the current state.

The 3 MUXes with the blue state name operate as follows: They output the next state if the current state is the one written on them in the blue color. For example, if the current state is PAUSED and trig = '1', the next state is COUNTING and so on.

The actual next state is chosen via a MUX that has its select as the current state.

The outputs are chosen based on the current state (Moore machine), with the "=" block being a comparator. The reset input resets the Q output of the register to the IDLE state.

6.3 Simulation Waveform

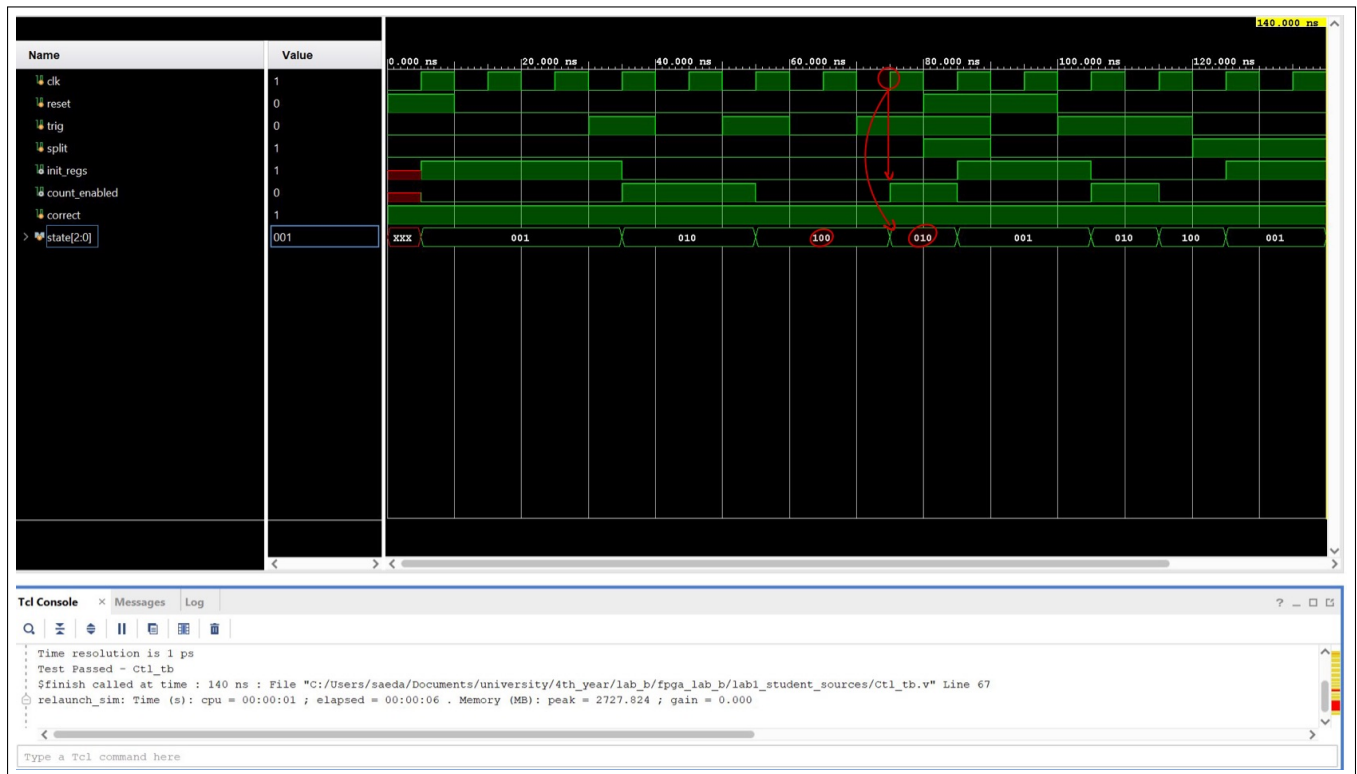


Figure 20: Control FSM Simulation Waveform (Annotated)

Waveform Explanation: As we can see, we got the Test Passed message on the console and correct stayed at '1' until the end of the simulation.

The state machine is behaving exactly as we want, according to the FSM diagram given to us in the task document.

To pinpoint one case and give an example: Looking at the marked clock edge on the wave diagram, the previous state was $100_b = PAUSED$ and the output was 00_b , trig got asserted during the previous cycle. The new state is now $010_b = COUNTING$, count_enabled switches to '1' and init_regs stays at '0'. This matches the Ctl state machine perfectly.

7 Debouncer

7.1 Block Diagram

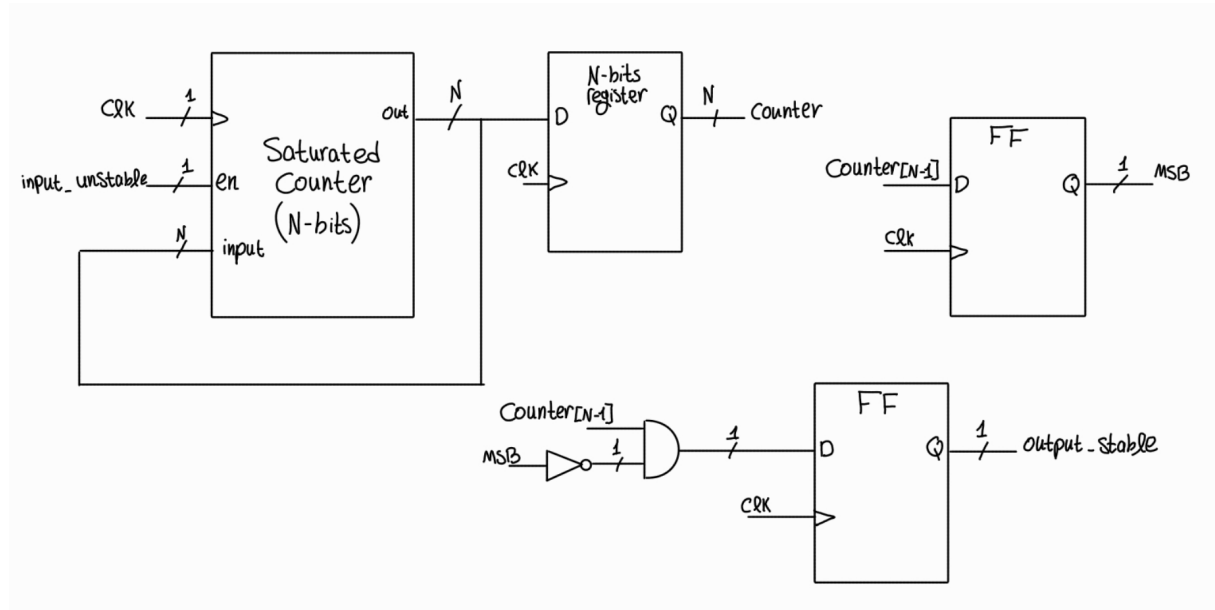


Figure 21: Debouncer Block Diagram

Block Diagram Explanation: Our design contains a saturated N-bit counter, which can increment to a maximum value of 2^{N-1} or decrement to a minimum value of 0 without wrap-around, two FFs and one N-bit register. As input we get the unstable input, which is connected directly to the counter, every clock rise in which the unstable input is '1' the counter is incremented, and decremented when '0'. The counter output is connected to an N-bit register. The first flip-flop samples the counter MSB at each clock rise. the second flip-flop, which generates the stable output, implementing the following logic:

$$output_stable(t) = \overline{MSB(t-1)} \& counter[N-1](t-1) = \overline{counter[N-1](t-2)} \& counter[N-1](t-1)$$

In other words, `output_stable` is '1' in the clock cycle where the MSB of the counter changes its value from '0' to '1'.

7.2 Counter Value Tradeoff

Question: Discuss the tradeoff between choosing high and low values for the counter, and how is this related to the low-pass-filter debouncing approach we saw in the previous experiments.

Answer: The COUNTER_BITS parameter determines the duration of the debounce interval. Increasing its value enlarges the counter size, which improves immunity to noise and contact bouncing, but also results in a longer latency before the system responds to a button press. Conversely, reducing this value shortens the response time, but increases the likelihood of false triggering caused by unstable or noisy input signals.

This tradeoff is equivalent to the low-pass-filter-based debouncing approach. A lower cutoff frequency averages the input over a longer time, suppressing bouncing at the cost of increased delay, while a higher cutoff frequency allows faster response with reduced noise suppression. Similarly, a larger counter value requires longer input stability, whereas a smaller counter value reacts faster but is more sensitive to noise.

8 Seven Segment Display (Seg_7_Display)

8.1 Clock Division Selection

Question: Describe your choice of bits for clock division and justify your answer analytically.

Answer:

The provided clock is a 100MHz clock, which means it ticks every 10ns. The clkdiv register provided in the design increments its value each cycle, so we can conclude a formula for the time it takes for each bit to change:

$$\text{the } n^{\text{th}} \text{ bit changes every } 10 \cdot 2^n \text{ ns}$$

Since in our design there are 4 numbers to display we need to select two bits so we can represent them. The total refresh time of the 4 digits should be between 1ms and 16ms so human eyes can see them without blinking effect. If we select the [18:17] bits, the number they represent will change each 1.31ms (bit 17 changes every $10 \cdot 2^{17} \text{ ns} = 1.31 \text{ ms}$) and in total, the refresh time for the two bits will be $4 \cdot 1.31 \text{ ms} = 5.24 \text{ ms}$, which satisfies the requirement.

9 Stopwatch

9.1 Block Diagram

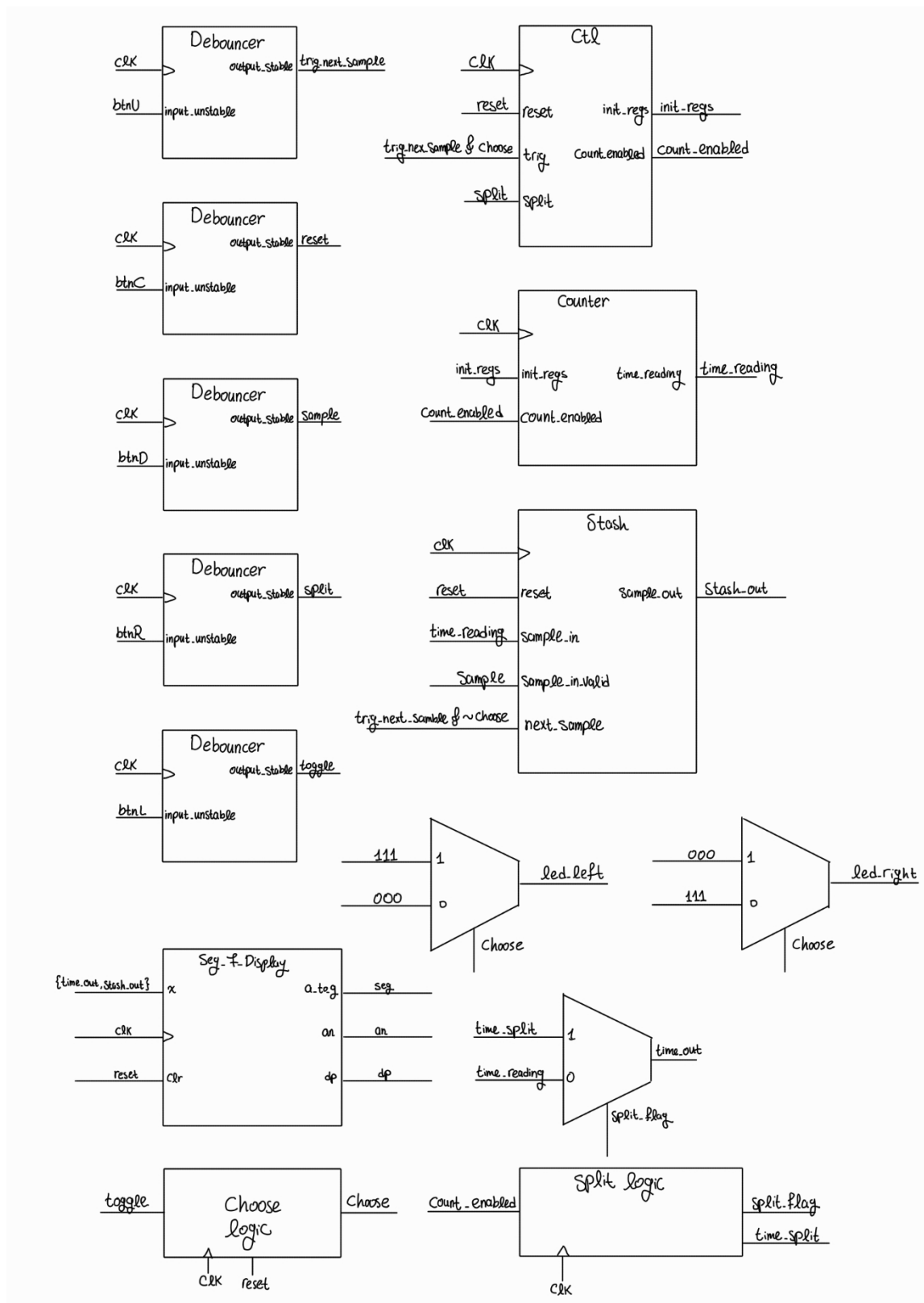


Figure 22: Stopwatch Top-Level Block Diagram

Block Diagram Explanation: The stopwatch design contains the following modules:

1. 5 Debouncers: One debouncer for each button, to make the button press stable.
2. Ctl: State machine to control the stopwatch
3. Counter: Counter module.
4. Stash: Stash module
5. Seg_7_Display: Connected the stash and counter outputs to the display.
6. 2 MUXes to control the leds
7. MUX to choose between time_reading and time_split
8. choose logic: A synchronized block to toggle the control between the counter and the stash, it receives as input the toggle signal in addition to clk and reset, the output is a 1-bit signal that indicates if we are using the counter or the stash. The signal is initialized to choose the counter.(see Appendix .1, Listing 1)
9. split logic: A synchronized block to implement the split logic. It generates a flag once the toggle button is pressed, the stopwatch display freezes while the counter is still counting in the background, each additional split press will update the shown time. All the logic works only when count_enabled signal is '1', which indicates that the ctl in the counting state, once we are outside that state, the split_flag is reset to '0'.(see Appendix .1, Listing 2)

9.2 Elaborated Design Schematic

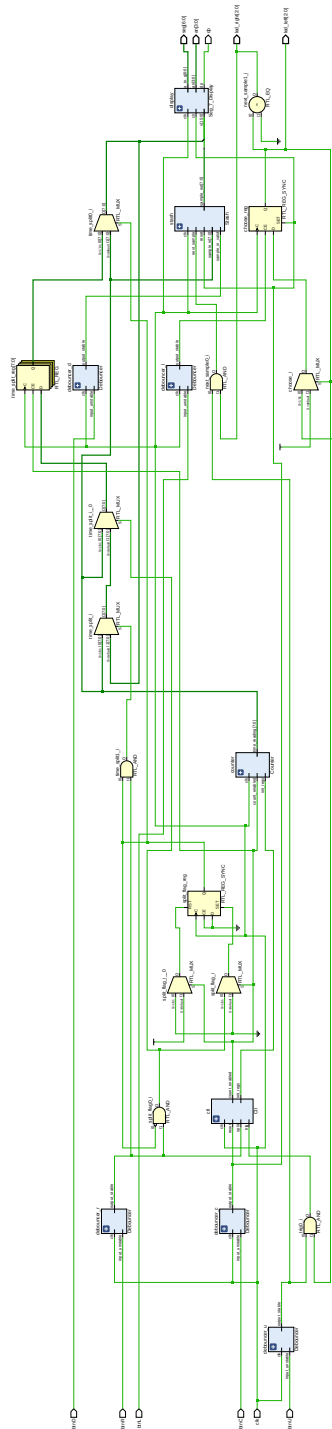


Figure 23: Elaborated Design Schematic

9.3 Synthesized Design Comparison

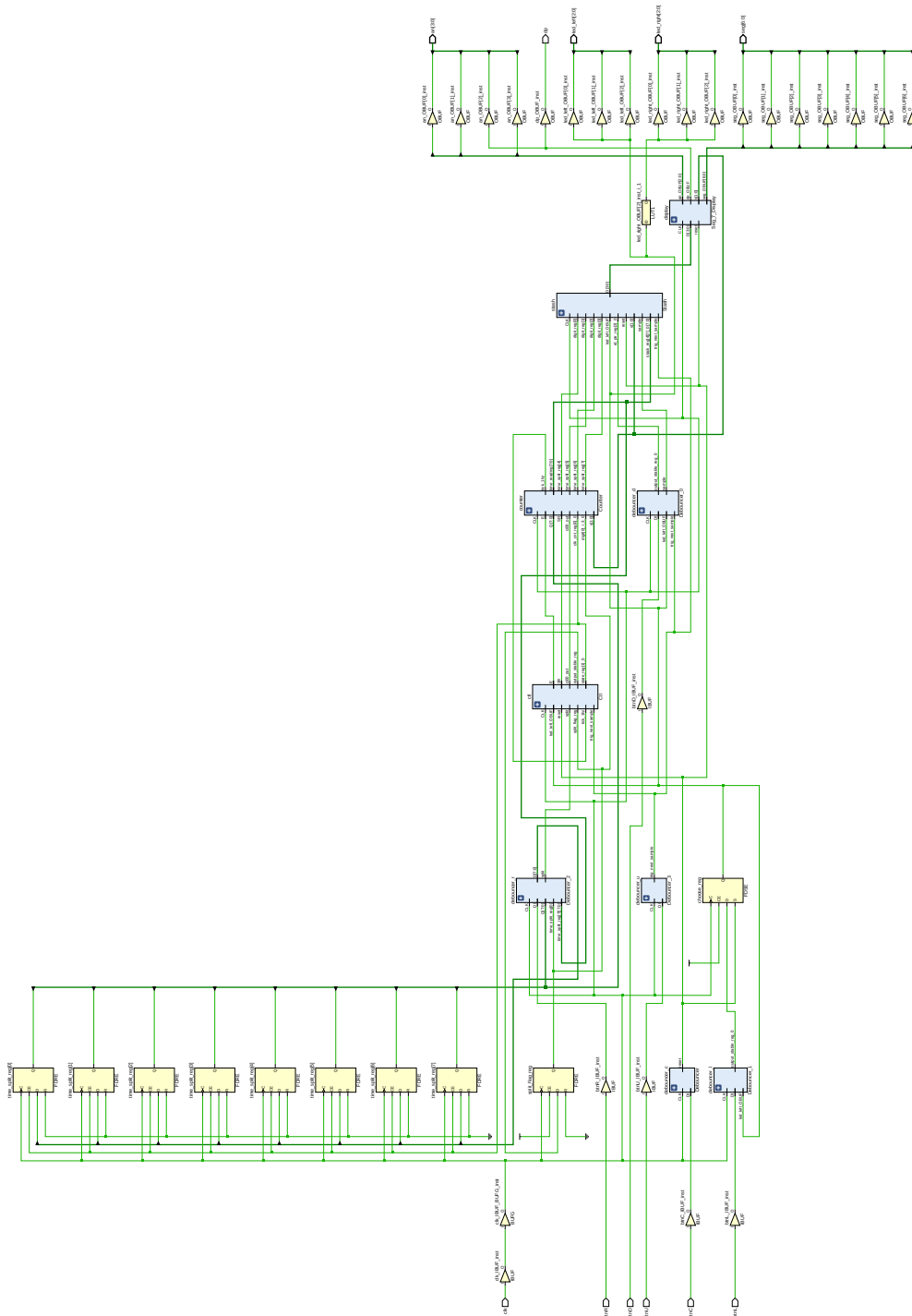


Figure 24: Synthesized Design Schematic

Question: Describe the differences that appeared compared to the elaborated design.

Differences: In the elaborated design we see the logical structure of the design as written in Verilog, before any optimization, we can see how modules, registers and logic

blocks are connected based on the Verilog code we wrote. In the synthesized design schematic, we see the actual hardware resourced that the FPGA chip will use to implement the Verilog code.

10 Timing summary

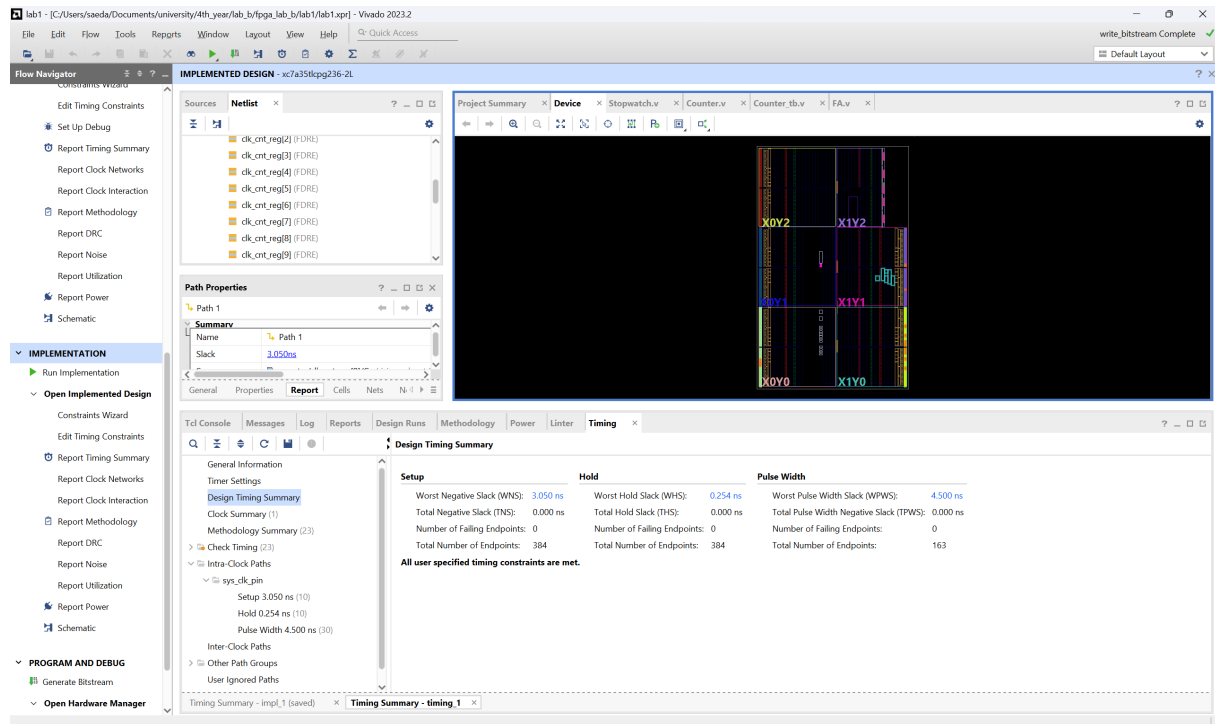


Figure 25: Timing summary

As we can see in Figure 25, our design meets all the timing constraints. To calculate the longest/critical path delay, we will use the following formula:

$$\text{longest_path} = \text{clock_period} - \text{setup_min_slack} \quad (1)$$

In our design, the clock frequency is 100MHz so the clock period is equal to 10ns. In Figure 25 we can see that the min slack is 3.050ns. Plugging those numbers in Equation 1 we get:

$$\text{longest_path} = 10\text{ns} - 3.050\text{ns} = 6.95\text{ns}$$

The longest_path above includes the logic delay in addition to setup time.

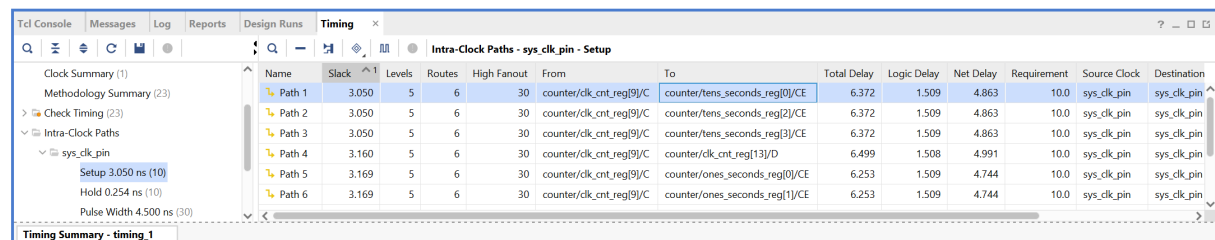


Figure 26: Timing summary

In Figure 26 we can see that the delay of the longest path (the path with the minimum slack) is:

$$\text{logic_delay} = 6.372\text{ns}$$

Appendix

.1 Verilog Implementations

```
1  always @(posedge clk) begin
2      if (reset) begin
3          choose <= COUNT;
4      end else if (toggle) begin
5          if (choose == COUNT) begin
6              choose <= STASH;
7          end else begin
8              choose <= COUNT;
9          end
10     end else begin
11         choose <= choose;
12     end
13 end
```

Listing 1: Choose logic

```
1  always @(posedge clk) begin
2      if (count_enabled) begin
3          if (~split_flag & split) begin
4              split_flag <= 1'b1;
5              time_split <= time_reading;
6          end else if (split_flag & split) begin
7              time_split <= time_reading;
8          end else if (split_flag) begin
9              time_split <= time_split;
10         end else begin
11             time_split <= time_reading;
12         end
13     end else begin
14         split_flag <= 1'b0;
15     end
16 end
```

Listing 2: split handling logic